



GLOSSÁRIO TÉCNICO • INTELIGÊNCIA ARTIFICIAL

Large Language Models

Da Inteligência Artificial aos Modelos GPT: conceitos,
arquitetura e o funcionamento de um LLM

IA • MACHINE LEARNING • DEEP LEARNING • TRANSFORMERS • GPT

Matheus Dapper
Vitória Aparecida Vendausen

2026
13 de agosto

Sumário

Pergunta 1 — O que caracteriza um Large Language Model?	1
Pergunta 2 — Qual a diferença entre IA, Machine Learning e Deep Learning?	2
Pergunta 3 — Quais são as principais etapas do treinamento de um LLM?	3
Pergunta 4 — Qual é o fluxo geral de funcionamento de um modelo de linguagem?	4
Pergunta 5 — Como os modelos GPT estão organizados?	6

Pergunta 1. O que caracteriza um Large Language Model?

LLM (Large Language Model)

Aplicação específica de deep learning: uma rede treinada em enormes volumes de texto para prever a próxima palavra, o que resulta na fluência human-like. A maioria dos LLMs modernos se baseia na arquitetura transformer, introduzida no artigo de 2017 “*Attention Is All You Need*” (<https://arxiv.org/abs/1706.03762>).

O que caracteriza um LLM, além da escala, é o mecanismo de **attention**, responsável por ponderar dinamicamente o contexto em cada previsão.

Attention — modelo mental: $C \times P = I$

O mecanismo de atenção pondera o contexto: cada token do contexto entra como um **Conceito** (seu conteúdo), recebe um **Peso** proporcional à sua relevância para o token atual, e a soma ponderada resulta na **Importância** — o contexto condensado que alimenta a predição seguinte.

```
// C x P = I -> Conceito x Peso = Importância

para cada token_atual no contexto:
  para cada token_i no contexto:
    C[i] = valor(token_i)                // Conceito: conteúdo do token
    P[i] = relevancia(token_atual, token_i) // Peso: quanto importa agora

  P = softmax(P)                        // normaliza: soma dos pesos = 1

  I = 0
  para cada token_i:
    I += C[i] * P[i]                    // Importância: contexto ponderado

  saida[token_atual] = I
```

Na notação formal:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) \cdot V,$$

onde V é o Conceito, $\text{softmax}(QK^T/\sqrt{d_k})$ é o Peso, e o produto é a Importância. Q (“o que eu procuro”) e K (“o que eu ofereço”) existem apenas para calcular o Peso — a similaridade entre ambos é o que gera a distribuição de atenção.

Observação

P é recalculado a cada token e a cada camada. É essa dinamicidade que distingue attention de um peso sináptico comum, que permanece fixo após o treinamento.

Pergunta 2. Qual a diferença entre IA, Machine Learning e Deep Learning?

IA (Inteligência Artificial)

Campo mais amplo, que engloba qualquer sistema capaz de executar tarefas que exigiriam inteligência humana: raciocínio, aprendizado, percepção e compreensão de linguagem. Abrange desde sistemas baseados em regras explícitas até redes neurais.

Machine Learning

Subcampo da IA em que a máquina infere as regras a partir dos dados, em vez de recebê-las codificadas manualmente por um programador. O treinamento é o processo de ajustar parâmetros do modelo para minimizar o erro em relação aos exemplos observados.

Deep Learning

Subcampo do Machine Learning que utiliza redes neurais com muitas camadas. Essa profundidade permite aprender representações hierárquicas diretamente dos dados brutos, dispensando a engenharia manual de features.

Relação entre os quatro

IA $\supset$ Machine Learning $\supset$ Deep Learning $\supset$ LLM

Não há contradição real entre essa hierarquia e o fato de a arquitetura transformer estar presente em LLMs, mas sim dois tipos distintos de relação convivendo no mesmo glossário:

- **IA $\supset$ ML $\supset$ DL $\supset$ LLM** — contenção estrita, e continua válida. Mesmo os LLMs recorrentes ou convolucionais são redes neurais profundas, logo permanecem dentro de Deep Learning.
- **Transformer $\cap$ LLM** — interseção. “Transformer” designa uma arquitetura, não um nível da hierarquia; ela ocupa posição transversal, podendo aparecer dentro ou fora do domínio da linguagem.

Erro a evitar

Encaixar “transformer” como uma camada da hierarquia, entre Deep Learning e LLM. Ele não é uma camada — é uma escolha de arquitetura.

Pergunta 3. Quais são as principais etapas do treinamento de um LLM?

1. Preparação dos dados

Coleta do corpus, tokenização do texto em unidades menores (tokens), criação dos pares entrada/alvo por sliding window e conversão dos tokens em embeddings vetoriais, somados à codificação posicional que informa a ordem das palavras.

2. Pré-treinamento (pretraining)

Treino auto-supervisionado sobre grandes volumes de texto não rotulado, com um único objetivo: prever a próxima palavra. O rótulo é o próprio texto deslocado em uma posição, o que dispensa anotação humana. É a etapa mais custosa em dados e computação, e seu resultado é o *foundation model*: um modelo com capacidade geral de linguagem, mas sem especialização.

3. Fine-tuning

Treino adicional sobre um dataset menor e rotulado, para especializar o modelo em uma tarefa. Duas variantes principais:

- **Instruction fine-tuning** — pares instrução/resposta, produzindo um assistente capaz de seguir comandos.
- **Classification fine-tuning** — exemplos com rótulos de classe (ex.: spam / não-spam).

4. Alinhamento (RLHF / DPO)

Etapa opcional que ajusta o modelo às preferências humanas quanto a utilidade e segurança. É o que diferencia um modelo apenas instruído de um produto final de conversação.

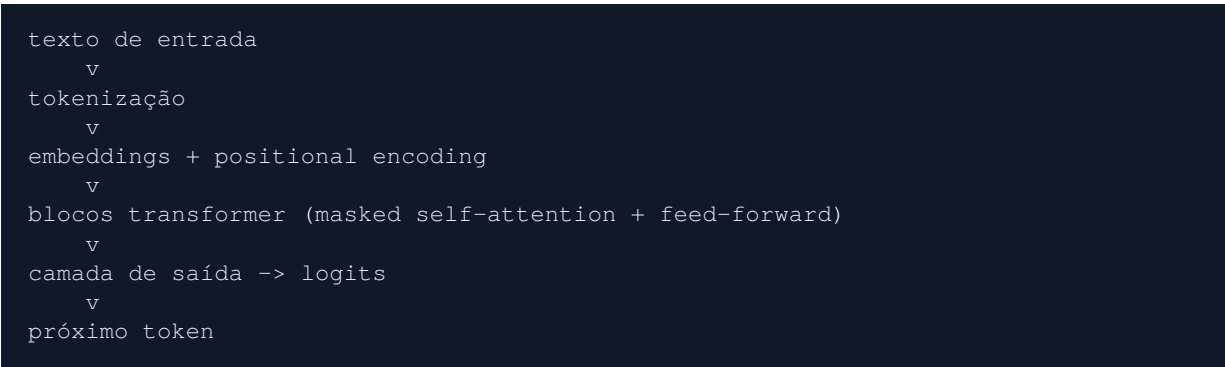
Pergunta 4. Qual é o fluxo geral de funcionamento de um modelo de linguagem?

Arquitetura Transformer

Distinção necessária: encoder e decoder são componentes estruturais, não etapas de treinamento.

A arquitetura encoder-decoder original, do artigo “*Attention Is All You Need*” (2017), foi projetada para tradução automática — o encoder processa o texto de origem e o decoder gera o texto de destino. GPT e a maioria dos LLMs modernos, porém, são **decoder-only**: dispensam o encoder por não haver uma sequência de origem a ser codificada separadamente.

Fluxo de dados (forward pass)



Esse fluxo ocorre tanto no treinamento quanto na inferência — não constitui uma sequência de fases de aprendizado.

Por que decoder-only funciona

Sem encoder, o modelo utiliza apenas masked self-attention, que impede cada posição de acessar tokens futuros. É precisamente essa máscara que torna a previsão da próxima palavra um objetivo de treinamento válido: o modelo nunca vê a resposta que deve prever (página 7 para mais informações).

Variantes arquiteturais

O transformer original é composto por blocos idênticos empilhados (N camadas), organizados em duas famílias de componentes. Modelos distintos utilizam uma ou ambas:

Modelo	Arquitetura	Objetivo de treino	Natureza
Transformer (2017)	Encoder-decoder	Tradução (seq2seq)	Geração condicionada
GPT	Decoder-only	Prever o próximo token	Autorregressivo
BERT	Encoder-only	Masked language modeling	Não autorregressivo

O GPT dispensa o encoder por não haver sequência de origem a codificar separadamente. Já o BERT prevê palavras mascaradas no interior da frase, com acesso ao contexto de ambos os lados — ele compreende texto, mas não gera continuações. O fluxo descrito a seguir é o do GPT.

O laço autorregressivo

O ponto central: um LLM não produz uma frase de uma só vez. Ele gera um token por vez, e o texto gerado até então retorna como entrada da passagem seguinte. Cada token exige um forward pass completo — gerar 50 tokens significa 50 passagens pela rede.

```

texto de entrada
  v
tokenização
  v
embeddings + positional encoding
  v
N blocos transformer (masked self-attention + feed-forward)
  v
logits (vetor do tamanho do vocabulário)
  v
softmax -> distribuição de probabilidade
  v
estratégia de decodificação -> token escolhido
  v
anexa o token ao contexto ----+
  ^                               |
  +----- repete -----+

```

Da saída da rede ao token

A rede não emite uma palavra, mas um vetor de logits com uma pontuação por item do vocabulário. O softmax converte esses valores em probabilidades, e apenas então uma estratégia de decodificação seleciona o token:

- **Greedy** — escolhe sempre o de maior probabilidade (determinístico).
- **Temperature** — achata ou acentua a distribuição, controlando a aleatoriedade.
- **Top-k / top-p** — restringe a amostragem aos candidatos mais prováveis.

É por isso que o mesmo prompt pode produzir respostas diferentes: o modelo é determinístico, a amostragem não é.

Janela de contexto

O laço não é ilimitado. O modelo comporta um número máximo de tokens visíveis simultaneamente — a *context window*, que define quanta informação ele mantém acessível durante a geração. Tokens que ultrapassam esse limite deixam de influenciar as previsões seguintes.

Pergunta 5. Como os modelos GPT estão organizados?

Base arquitetural

Os GPTs utilizam como conceito de arquitetura o padrão Transformer — porém adotado apenas parcialmente e de forma proposital, voltada à geração de texto.

Conforme o livro: o GPT foca na porção decoder da arquitetura transformer original, sendo projetado para tarefas que exigem gerar textos. Isso inclui tradução automática, sumarização, escrita de ficção, escrita de código e outras.

Precisão importante

O GPT não é literalmente “metade” do transformer. Ele aproveita a porção decoder, mas com uma alteração estrutural — o decoder original possuía duas camadas de atenção, uma self-attention e uma cross-attention voltada à saída do encoder. Sem encoder, a cross-attention deixa de existir. O bloco do GPT é, portanto, mais simples que o decoder original, e não uma cópia dele.

O nome como definição

- **Generative** — produz texto novo, token a token.
- **Pre-trained** — passa por pré-treinamento em grande corpus antes de qualquer especialização.
- **Transformer** — construído sobre a arquitetura de 2017.

Fundamento teórico

O modelo se baseia no artigo “*Improving Language Understanding by Generative Pre-Training*” (<https://mng.bz/x2qg>), de Radford et al.

Correção conceitual necessária

Dataset não é um campo físico de processamento ou armazenamento — isso descreve datacenter ou infraestrutura de compute. Um dataset é simplesmente um conjunto de dados; no caso do GPT, o corpus de texto usado no treinamento. A relação causal também se inverte em relação à formulação inicial: não é “possuir mais datasets” que possibilita a alta geração de texto, mas sim o volume e a diversidade do corpus combinados ao número de parâmetros. O GPT-3 foi treinado com aproximadamente 300 bilhões de tokens.

Versatilidade: zero-shot e few-shot

Segundo o livro, os modelos GPT — projetados e treinados primariamente para completar texto — demonstram versatilidade notável, sendo capazes de executar tarefas em regime zero-shot e few-shot:

- **Zero-shot** — generaliza para tarefas inteiramente novas, sem qualquer exemplo prévio.
- **Few-shot** — aprende a partir de um número mínimo de exemplos fornecidos pelo usuário como entrada.

Armadilha de nomenclatura a registrar

Apesar do termo *learning*, nenhum dos dois envolve treinamento. Os pesos permanecem congelados; os exemplos entram no prompt, em tempo de inferência.

Comportamento emergente

O GPT foi treinado com um único objetivo — prever a próxima palavra —, mas exibe capacidades que jamais foram programadas explicitamente, como tradução entre idiomas. Essas habilidades não constavam do objetivo de treino: emergiram da escala. É o que conecta a simplicidade do “apenas prever o próximo token” à versatilidade real observada no modelo.